

buscar_usuario_trainer.js — Fundamentos

Explicación detallada de la pantalla de búsqueda de alumnos (GymBro App)

Es la pantalla de **inicio** para los entrenadores (primera ruta del grupo “Entrenador” en `app/index.tsx`, así que es lo primero que ven al loguearse). Le permite al entrenador buscar entre sus alumnos y acceder a dos acciones: ver sus datos o editar su rutina.

1. Estado local (líneas 27-30)

```
const [selectedUser, setSelectedUser] = useState(null);
const [filtro, setFiltro] = useState('');
```

Dos estados simples, sin objetos anidados:

- **selectedUser**: guarda el **id** del alumno que el entrenador tocó en la lista (no el objeto completo, solo el id — patrón liviano, ya visto en `routes.js` con `exercisId`). `null` significa “nadie seleccionado”.
- **filtro**: el texto que el entrenador escribe en el buscador.

2. normalizar — función auxiliar repetida en el proyecto (líneas 20-24)

```
const normalizar = (texto) =>
  texto
    .normalize('NFD')
    .replace(/[\p{Diacritic}]/g, '')
    .toLowerCase();
```

Ya se ve la misma idea en `editar_rutina_trainer.js` para el buscador de ejercicios — acá es la misma lógica, redefinida localmente en este archivo (no está extraída a un helper compartido, por eso aparece duplicada en ambos lugares; es un candidato razonable a mover a un archivo de utilidades si quisieran evitar la repetición).

- `.normalize('NFD')` descompone los caracteres acentuados en su letra base + el acento como carácter separado (ej. “á” se convierte en “a” + el símbolo de tilde combinante).
- `.replace(/[\p{Diacritic}]/g, "")` elimina esos símbolos de acento sueltos con una expresión regular que cubre el rango Unicode de “diacríticos combinantes”.
- `.toLowerCase()` uniforma mayúsculas/minúsculas.

El resultado: buscar “gaetmank” encuentra a “Gaetmank” sin importar tildes ni mayúsculas, comparando texto ya normalizado de ambos lados.

3. El pipeline de filtrado (líneas 34-40)

```
const usuariosFiltrados = users
  .filter((user) => !user.isTrainer)
  .filter((user) =>
    normalizar(`${user.firstName} ${user.lastName} ${user.email}`)
      .includes(normalizar(filtro))
  );
```

No es un `useState` derivado ni un `useMemo`, es una variable calculada directamente en el cuerpo del componente, en **cada render**. Como React vuelve a ejecutar la función del componente cada vez que

cambia filtro (por setFiltro en el onChangeText del buscador), este filtro se recalcula automáticamente sin necesitar ningún efecto ni memoización — con un array de 5 usuarios de ejemplo, el costo de recalcularlo en cada tecla es insignificante, así que no hace falta optimizar con useMemo acá.

Dos pasos encadenados:

- Excluye entrenadores (!user.isTrainer).
- Filtra por coincidencia de texto en nombre/apellido/email concatenados.

4. Renderizado de la lista con .map() (líneas 90-166)

```
{usuariosFiltrados.map((user) => {  
  const isSelected = selectedUser === user.id;  
  return (  
    <TouchableOpacity  
      key={user.id}  
      style={[styles.userCard, isSelected && styles.selectedCard]}  
      onPress={() => setSelectedUser(user.id)}  
      activeOpacity={0.8}  
    >  
      ...  
    </TouchableOpacity>  
  );  
})}
```

- `key={user.id}` es obligatorio en toda lista renderizada con `.map()` en React — le permite al framework identificar qué elemento es cuál entre renders, para actualizar solo lo que cambió en vez de re-dibujar toda la lista.
- **isSelected se recalcula por cada card**, comparando el id de esa tarjeta puntual contra el único `selectedUser` global. Es el mismo patrón de “un solo estado controla el resaltado de una lista de opciones” que ya se ve en `OptionChips` de `signup.js` (`selected === option`), aplicado ahora a tarjetas en vez de pastillas.
- Tocar una tarjeta llama `setSelectedUser(user.id)` — no hay “deseleccionar” tocando de nuevo la misma tarjeta (no hay un toggle), simplemente cambia cuál es el seleccionado. Si ya había otro seleccionado, al tocar uno nuevo el anterior se deselecta automáticamente sin código extra, porque solo hay **un** valor de `selectedUser` en todo el componente.
- `activeOpacity={0.8}`: prop nativa de `TouchableOpacity` que controla qué tan transparente se pone al presionar (por defecto es 0.2; acá lo suavizan a 0.8, un efecto más sutil).

5. Renderizado condicional de las acciones (líneas 130-162)

```
{isSelected && (  
  <View style={styles.actionsContainer}>  
    <TouchableOpacity onPress={() => navigation.navigate('ver_datos_trainer', { user })}>  
      ...Ver datos...  
    </TouchableOpacity>  
    <TouchableOpacity onPress={() => navigation.navigate('editar_rutina_trainer', { user })}>  
      ...Editar rutina...  
    </TouchableOpacity>  
  </View>  
)}
```

Los dos botones de acción **solo existen en el árbol nativo cuando esa tarjeta puntual está seleccionada** — no están ocultos con opacidad ni `display: none`, directamente no se renderizan (`isSelected && (...)`, mismo patrón `&&` que el mensaje de error de `login.js`). Esto reduce trabajo de renderizado: de 3 alumnos en pantalla, solo la tarjeta activa dibuja sus dos botones extra.

6. Navegación con parámetros — lo nuevo de esta pantalla

```
onPress={() => navigation.navigate('ver_datos_trainer', { user })}
onPress={() => navigation.navigate('editar_rutina_trainer', { user })}
```

Es la **primera vez** que aparece `navigate()` con un **segundo argumento**. Hasta ahora (`login.js` → `signup`) solo se navegaba sin pasar datos. Acá, el segundo argumento es un objeto de **parámetros de ruta**: React Navigation se lo entrega a la pantalla destino a través de la prop `route`.

Esto es exactamente lo que consume `editar_rutina_trainer.js`:

```
export default function EditRoutineScreen({ navigation, route }) {
  const editedUser = route?.params?.user; // <- el "user" que se pasa desde acá
```

Es el mecanismo estándar de React Navigation para **pasar datos entre pantallas sin usar Context ni props manuales**: la pantalla de origen decide qué alumno se seleccionó, lo empaqueta en `{ user }`, y la pantalla destino lo recupera con `route.params.user`. El `?.` (optional chaining) en `route?.params?.user` protege contra el caso en que alguien entrara a esa pantalla sin haber pasado por acá — evita un error si `route.params` fuera `undefined`.

Por qué mandar el objeto completo (user) y no solo el id: como `users` es un array en memoria compartido por toda la app (no hay que “traerlo” de un servidor), sería igual de válido mandar solo `user.id` y que la pantalla destino llame `getUserById(id)`. Acá se optó por pasar el objeto completo directamente — ahorra una búsqueda extra, a costa de que si `users` cambiara entre que se navega y se usa el dato, la pantalla destino tendría una copia potencialmente desactualizada (no ocurre en este proyecto, pero es un matiz de diseño a mencionar si preguntan por qué no reusa `getUserById`).

7. Los estilos — layout basado en porcentajes

```
avatar: {
  width: '9%',
  aspectRatio: 1,
  borderRadius: 999,
  ...
},
```

Vale la pena señalar un detalle de estilo distinto a lo visto en `login.js/signup.js`: acá casi todos los tamaños (`minHeight: '7%'`, `padding: '3.5%'`, `width: '9%'`) están expresados en **porcentaje** en vez de píxeles fijos. Combinado con `aspectRatio: 1` en el avatar, esto hace que el círculo del avatar escale proporcionalmente al ancho de pantalla disponible, en vez de tener un tamaño fijo que se vería desproporcionado en pantallas muy chicas o muy anchas. `borderRadius: 999` es el truco típico para lograr un círculo perfecto: un valor de radio mayor a la mitad del ancho/alto simplemente lo recorta en círculo, sin necesitar calcular el valor exacto.

`isLandscape` (ya conocido) ajusta el header y el content en modo horizontal — en `contentLandscape` además fija un `maxWidth: 900` y `alignSelf: 'center'`, para que en pantallas anchas el listado no se estire de punta a punta sino que quede centrado con un ancho máximo legible.

8. Conexión con TrainerBottomNav (líneas 172-176)

```
<TrainerBottomNav
  activeScreen="search"
  isLandscape={isLandscape}
  navigation={navigation}
/>
```

Ya se documentó en detalle en `TrainerBottomNav.pdf`: acá se ve el ejemplo concreto de cómo esta pantalla se auto-identifica (`activeScreen="search"`) y reenvía `isLandscape/navigation` como props.

Resumen para el oral

“buscar_usuario_trainer.js es la pantalla de inicio del entrenador: filtra users en dos pasos (excluir entrenadores, buscar texto normalizado sin tildes) usando un cálculo derivado que se recalcula en cada render, sin useMemo. Cada tarjeta de alumno usa un único estado selectedUser para controlar su resaltado y mostrar condicionalmente sus dos acciones. Es la primera pantalla que navega con parámetros (navigation.navigate('ruta', { user })), pasando el objeto completo del alumno seleccionado a ver_datos_trainer y editar_rutina_trainer, que lo reciben a través de la prop route.params.”